Today 2:58 PM

Thanks for all the help, one last thing, could you help me make the page unit tests more in depth? Like checking that the page displays specific things

Worked for 10s

Definitely! You can make your page tests more detailed by checking not only that a route returns a `Page`, but also that the page contains the expected text and interface components.

For your Alternative Archive project, I'd test things like:

- Does the home page display the welcome message?
- Does it have the Search, Recommend, and About buttons?
- Does the Search page display Theme, Mood, and Genre?
- Does the Recommend page display Name, Artist, and the three category fields?
- Does the Result page display the recommended song or the no-matches message?
- Does the About page display your website description?

There is one important distinction: `assert_equal(type(index(test_state)), Page)` checks the page type, but it doesn't check the page's contents.

1. How to test page contents using only `assert_equal`

Drafter pages have a `.content` attribute, which is a list containing the things displayed on the page. You can test individual items using their positions in that list. Drafter's documentation confirms that components

and lists can be compared with `assert_equal`.

For example:

Python

```
test_page = index(test_state)

assert_equal(test_page.content[0], "Welcome to Alternative Archive!")
assert_equal(test_page.content[1], Button("Search", "search"))
```

This checks the actual welcome message and the Search button, including which route the button leads to.

2. Tests for each of your five pages

Put these at the bottom of your program, after your initial state and song library.

Home page

Your home page has four content items: a welcome message and three buttons.

Python

```
test_page = index(initial_state)

assert_equal(type(test_page), Page)
assert_equal(test_page.content[0], "Welcome to Alternative Archive!")
assert_equal(test_page.content[1], Button("Search", "search"))
assert_equal(test_page.content[2], Button("Recommend" ecommend"))
assert_equal(test_page.content[3], Button("About", "about_page"))
```

Search page

Your Search page has eight content items.

Python Run

```
test_page = search(initial_state)

assert_equal(type(test_page), Page)
assert_equal(test_page.content[0], "Find Your Song!")
assert_equal(test_page.content[1], "Theme")
assert_equal(test_page.content[3], "Mood")
assert_equal(test_page.content[5], "Genre")
assert_equal(test_page.content[7], Button("Search", "perform_search"))
```

This checks that all three category labels and the Search button are displayed.

Recommend page

This is a particularly useful one to test because it has several inputs.

Python Run

```
test_page = recommend(initial_state)

assert_equal(type(test_page), Page)
assert_equal(test_page.content[1], "Name:")
assert_equal(test_page.content[2], TextBox("name"))
assert_equal(test_page.content[3], "Artist:")
assert_equal(test_page.content[4], TextBox("artist"))
```

Python Run

```
assert_equal(test_page.content[5], "Theme:")
assert_equal(test_page.content[7], "Mood:")
assert_equal(test_page.content[9], "Genre:")
assert_equal(test_page.content[11], Button("Submit Song", "add_song"))
assert_equal(test_page.content[12], Button("Home", "index"))
```

About page

Your About page has a paragraph and a Home button.

Python Run

```
test_page = about_page(initial_state)

assert_equal(type(test_page), Page)
assert_equal(
    test_page.content[0],
    "Welcome to Alternative Archive! This was made as a project for a University Computer Science class with the
)
assert_equal(test_page.content[1], Button("Home", "index"))
```

Result page

For this test, use a state with exactly one matching song so that the random selection doesn't make your test unpredictable.

Python Run

```python
test_state = State(
    library=[
        Song("Everlong", "Foo Fighters",
             "Appreciation", "Upbeat", "Grunge")
    ],
    search_theme="Appreciation",
    search_mood="Upbeat",
    search_genre="Grunge",
    result_message="",
    submission_message=""
)

test_page = result(test_state)

assert_equal(type(test_page), Page)
assert_equal(test_page.content[0], "Your Song Is:")
assert_equal(test_page.content[1], "Everlong by Foo Fighters")
assert_equal(test_page.content[2], Button("Home", "index"))
```

You should also test the no-results version:

Python Run

```python
test_state = State(
    library=[],
    search_theme="Appreciation",
    search_mood="Upbeat",
    search_genre="Grunge",
```

Python Run

```
    result_message="",
    submission_message=""
)

test_page = result(test_state)

assert_equal(test_page.content[0], "Your Song Is:")
assert_equal(test_page.content[1], "No matching songs found.")
```

3. One extra test I'd recommend

You can also test whether the Recommend page displays the correct confirmation message after a song is added.

Python Run

```
test_state = State(
    library=[],
    search_theme="",
    search_mood="",
    search_genre="",
    result_message="",
    submission_message=""
)

test_page = add_song(
    test_state,
    "Everlong",
    "Foo Fighters",
```

Python Run

```
    "Appreciation",
    "Upbeat",
    "Grunge"
)

assert_equal(test_page.content[0],
             "Song successfully added to the archive!")
assert_equal(test_page.content[1], "Name:")
```

This tests both the message and that the user is returned to the Recommend page.